



CYBERSECURITY PASSWORD STRENGTH ANALYZER

Workshop Project Technical Report

Object-Oriented Programming with Python

Language	Python 3.x
Paradigm	Object-Oriented Programming
Domain	Cybersecurity / Password Analysis
Files	11 Python Files
Date	June 2026

Table of Contents

1. Project Overview

The Cybersecurity Password Strength Analyzer is a comprehensive, menu-driven Python application built entirely around Object-Oriented Programming (OOP) principles. The project demonstrates how real-world security concepts — password strength scoring, vulnerability detection, entropy calculation, and report generation — can be modelled using classes, inheritance, composition, and polymorphism.

The system allows users to register accounts, analyze passwords in depth, generate cryptographically-inspired strong passwords or passphrases, view historical analysis records, and export detailed security reports to text and CSV files — all through an interactive terminal interface.

1.1 Key Features at a Glance

Feature	Description	OOP Concept
Password Analysis	Multi-factor scoring: length, diversity, entropy, patterns	Encapsulation
Vulnerability Scan	Checks common passwords, dictionary words, keyboard walks	Composition
Password Generator	Random passwords and word-based passphrases	Encapsulation
User Management	Register users, track history per user	Inheritance
Report Generation	Formatted security reports with full audit trail	Composition
Data Persistence	Save/load all data as JSON files	Serialization
CSV Export	Export analysis records to spreadsheet-friendly CSV	Service Layer

1.2 Project Objectives

The system is designed to achieve the following goals:

- Analyze password strength using multiple factors — length, character variety, entropy, and pattern detection.
- Detect vulnerabilities: common password lists, dictionary words, repeated characters, and keyboard-walk patterns.
- Estimate password-cracking difficulty based on entropy and charset size.
- Generate strong random passwords and passphrases that meet all complexity requirements.
- Maintain per-user password analysis history with JSON persistence.
- Export security data to CSV files for external analysis.
- Demonstrate all core OOP pillars in a real-world cybersecurity context.

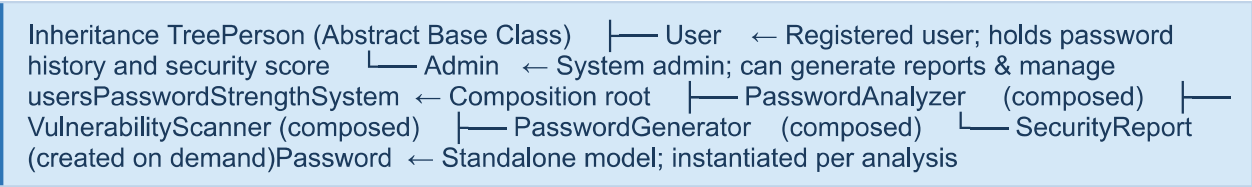
2. Project Architecture

The project follows a three-layer architecture with a clear separation of concerns. Each layer has a defined responsibility and communicates with adjacent layers through well-defined interfaces.

2.1 Directory & Layer Structure

Layer	Package / File	Responsibility
Entry Point	main.py	Menu-driven UI, user input/output, handlers
Service Layer	services/	AnalysisService, ReportService, ExportService — orchestration logic
Domain Models	models/	9 core classes — all OOP concepts live here
Utilities	utils/	helpers.py, exceptions.py — decorators, validators, custom errors
Data Store	data/, reports/, exports/	JSON files, report text files, CSV exports

2.2 Class Hierarchy



2.3 OOP Pillars Applied

Pillar	Where It Appears	Example
Abstraction	Person (ABC) defines display_details() and update_profile() as abstract	@abstractmethod in person.py
Encapsulation	All classes use double-underscore private attributes with @property	self.__password_text in Password
Inheritance	User and Admin both extend Person, calling super().__init__()	user.py / admin.py extend person.py
Polymorphism	display_details() behaves differently for User vs Admin at runtime	demonstrate_polymorphism() in main.py
Composition	PasswordStrengthSystem owns PasswordAnalyzer, VulnerabilityScanner, PasswordGenerator	password_strength_system.py

3. File-by-File Code Walkthrough

3.1 main.py — Application Entry Point & Menu Interface

Purpose: main.py is the application entry point. It creates the global system instance, starts the interactive menu loop, and routes user choices to the appropriate handler functions. It also includes a polymorphism demonstration function.

OOP & Python Concepts Demonstrated

- Composition — Creates global instances of PasswordStrengthSystem, AnalysisService, ReportService, and ExportService.
- Polymorphism — demonstrate_polymorphism() creates a User and an Admin, then calls display_details() on both, showing runtime polymorphic dispatch.
- Exception Handling — Each handler catches EmptyPasswordError, InvalidPasswordFormatError, ValueError, and KeyError with descriptive messages.
- Auto-load / Auto-save — System loads JSON data on startup and saves before exit.

Menu Options

Option	Handler	Description
1. User Registration	handle_user_registration()	Register a new user; validates user ID, name, and email
2. Analyze Password	handle_analyze_password()	Full pipeline: strength score, patterns, vulnerabilities, crack time
3. Generate Password	handle_generate_password()	Random password, passphrase, or 5 suggestions
4. View History	handle_view_history()	All records, user-specific, search, sorted, weak/strong/risky
5. Security Reports	handle_security_reports()	Generate and export reports in text and CSV format
6. Save Data	handle_save_data()	Serializes all data to JSON files
7. Load Data	handle_load_data()	Deserializes data from JSON files
8. Exit	sys.exit(0)	Auto-saves, then exits cleanly

Full Source Code

```
# =====  
# CYBERSECURITY PASSWORD STRENGTH ANALYZER  
# Main Entry Point - Menu-Driven Interface  
# Run with: python main.py  
# =====  
  
import sys, os  
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

```

from models.password_strength_system import PasswordStrengthSystem
from models.password_analyzer import PasswordAnalyzer
from services.analysis_service import AnalysisService
from services.report_service import ReportService
from services.export_service import ExportService
from utils.helpers import display_banner, display_menu, display_report_submenu,
    get_valid_input, clear_screen, validate_email, validate_name
from utils.exceptions import EmptyPasswordError, InvalidPasswordFormatError,
    WeakPasswordError, InvalidUserInputError

# Global system instances (Composition Root)
system = PasswordStrengthSystem()
analysis_service = AnalysisService(system)
report_service = ReportService(system)
export_service = ExportService(system)
current_user_id = None

def handle_user_registration():
    """Menu option 1: Register a new user."""
    global current_user_id
    try:
        user_id = get_valid_input(' Enter User ID : ', validator=lambda x:
len(x)>=2,
                                error_msg='User ID must be at least 2
characters.')
        name = get_valid_input(' Enter Full Name : ', validator=validate_name,
                                error_msg='Name must contain only letters and
spaces.')
        email = get_valid_input(' Enter Email : ', validator=validate_email,
                                error_msg='Please enter a valid email address.')
        user = system.register_user(user_id, name, email)
        current_user_id = user_id
        print(' ✓ Registration successful!')
        print(user.display_details())
    except ValueError as e:
        print(f' ⚠ Registration failed: {e}')

def handle_analyze_password():
    """Menu option 2: Analyze a password."""
    global current_user_id
    try:
        password = input(' Enter password to analyze: ')
        if not password: raise EmptyPasswordError()
        if current_user_id:
            assoc = input(f' Associate with user {current_user_id!r}? (y/n):
').strip().lower()
            uid = current_user_id if assoc == 'y' else None
        else:
            uid = None
        result = analysis_service.analyze(password, user_id=uid)
        print(analysis_service.display_analysis(result))
        if result.get('score', 0) <= 20:
            print(' ⚠ WARNING: This password is CRITICALLY WEAK!')
            suggestions = analysis_service.get_suggestions()
            for i, s in enumerate(suggestions, 1): print(f' {i}. {s}')
    except EmptyPasswordError as e:
        print(f' ⚠ Error: {e}')

def handle_generate_password():
    # Presents 3 sub-options: Random, Passphrase, 5 Suggestions
    # Delegates to analysis_service.generate_password() / generate_passphrase()

```

```
...

def handle_view_history():
    # Sub-menu: all records, user history, search, sorted, weak/strong/risky
    # Delegates to analysis_service.sort_by_score(), get_weak_passwords() etc.
    ...

def handle_security_reports():
    # Sub-menu: password report, user report, risk assessment, summary,
    #           CSV export, generate files
    ...

def demonstrate_polymorphism():
    """Demonstrates runtime polymorphism via Person hierarchy."""
    from models.user import User
    from models.admin import Admin
    persons = [User('DEMO-U1', 'Demo User', 'user@demo.com'),
               Admin('DEMO-A1', 'Demo Admin', 'admin@demo.com', admin_id='ADM-001')]
    for person in persons:
        print(person.display_details())    # Polymorphic call

def main():
    global current_user_id
    clear_screen(); display_banner()
    try: system.load_data()
    except Exception: pass
    while True:
        display_menu()
        choice = input(' Enter your choice (1-8): ').strip()
        if choice == '1': handle_user_registration()
        elif choice == '2': handle_analyze_password()
        elif choice == '3': handle_generate_password()
        elif choice == '4': handle_view_history()
        elif choice == '5': handle_security_reports()
        elif choice == '6': system.save_data()
        elif choice == '7': system.load_data()
        elif choice == '8': system.save_data(); sys.exit(0)
        input('\n Press Enter to continue...')

if __name__ == '__main__':
    main()
```

3.2 person.py — Abstract Base Class (Abstraction + Encapsulation)

Purpose: person.py defines the abstract base class Person using Python's abc module. It establishes the contract that all person-like entities in the system must satisfy, while privately storing core attributes using name-mangling for encapsulation.

OOP Concepts Demonstrated

- Abstraction — Two @abstractmethod decorators (display_details, update_profile) force subclasses to provide their own implementation. Person cannot be instantiated directly.
- Encapsulation — Private attributes use double-underscore name mangling (self.__user_id, self.__name, self.__email). External code can only access them via @property decorators.
- Properties with validation — email setter requires '@' in the value; name setter rejects empty strings.
- Magic Methods — __str__ and __repr__ give human-readable and debug-friendly string representations.
- Serialization — to_dict() converts the object to a JSON-serializable dictionary.

Full Source Code

```
# =====
# Person - Abstract Base Class
# Cybersecurity Password Strength Analyzer
# =====

from abc import ABC, abstractmethod
from datetime import datetime

class Person(ABC):
    """
    Abstract base class for all persons in the system.
    Demonstrates: Abstraction, Encapsulation, Constructor,
                  Magic Methods (__str__, __repr__)
    """

    def __init__(self, user_id: str, name: str, email: str):
        self.__user_id = user_id      # Private (name-mangled)
        self.__name     = name
        self.__email    = email
        self._created_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    @property
    def user_id(self) -> str: return self.__user_id

    @property
    def name(self) -> str: return self.__name
    @name.setter
    def name(self, value: str):
        if not value or not value.strip():
            raise ValueError('Name cannot be empty.')
        self.__name = value.strip()

    @property
    def email(self) -> str: return self.__email
    @email.setter
    def email(self, value: str):
```

```
        if not value or '@' not in value:
            raise ValueError('Invalid email address.')
        self.__email = value.strip()

    @abstractmethod
    def display_details(self) -> str: pass    # Must override in subclass

    @abstractmethod
    def update_profile(self, **kwargs) -> None: pass

    def __str__(self) -> str: return
    f'{self.__class__.__name__}(id={self.__user_id}, name={self.__name})'
    def __repr__(self) -> str: return
    f'{self.__class__.__name__}(user_id={self.__user_id!r}, name={self.__name!r})'

    def to_dict(self) -> dict:
        return { 'user_id': self.__user_id, 'name': self.__name,
                  'email': self.__email, 'created_at': self.__created_at,
                  'type': self.__class__.__name__ }
```

3.3 user.py — Registered User (Inheritance + Polymorphism)

Purpose: User extends Person. It adds password history tracking and a personal security score. It overrides display_details() to show a user-specific terminal dashboard, demonstrating polymorphism.

OOP Concepts Demonstrated

- Inheritance — Calls `super().__init__(user_id, name, email)` to reuse Person's constructor and private attribute initialization.
- Polymorphism — `display_details()` renders a USER PROFILE box; its output is entirely different from Admin's version of the same call.
- Encapsulation — `self.__password_history` (list) and `self.__security_score` (float, 0–100) are private; `security_score` has a validated setter.
- Magic Methods — `__len__` returns how many passwords the user has analyzed; `__str__` produces a one-line summary.
- Serialization — `to_dict()` and `from_dict()` enable complete JSON persistence round-trips.

Full Source Code

```
# =====
# User - Derived from Person
# Cybersecurity Password Strength Analyzer
# =====

from datetime import datetime
from models.person import Person

class User(Person):
    """
    Registered user who can analyse passwords.
    Demonstrates: Inheritance, Polymorphism, Encapsulation.
    """

    def __init__(self, user_id, name, email, password_history=None,
security_score=0.0):
        super().__init__(user_id, name, email)
        self.__password_history = password_history if password_history else []
        self.__security_score = security_score

    @property
    def password_history(self) -> list: return list(self.__password_history)
    @property
    def security_score(self) -> float: return self.__security_score
    @security_score.setter
    def security_score(self, value): self.__security_score = max(0.0, min(100.0,
value))

    def display_details(self) -> str:
        return (
            f'
            f'          USER PROFILE
            f'          ||\n'
            f' ID      : {self.user_id:<28}||\n'
            f' Name   : {self.name:<28}||\n'
            f' Score  : {self.__security_score:<28.1f}||\n'
            f'
        )
```

```
def update_profile(self, **kwargs) -> None:
    if 'name' in kwargs: self.name = kwargs['name']
    if 'email' in kwargs: self.email = kwargs['email']

def analyze_password(self, password_text: str, analysis_result: dict) -> None:
    record = { 'password_masked': password_text[:2] + '*' *
(len(password_text)-2),
              'score': analysis_result.get('score', 0),
              'strength': analysis_result.get('strength', 'Unknown'),
              'analyzed_at': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
              'details': analysis_result }
    self.__password_history.append(record)
    self.__security_score = analysis_result.get('score', self.__security_score)

def view_history(self) -> list: return list(self.__password_history)

def __len__(self) -> int: return len(self.__password_history)
def __str__(self) -> str: return f'User({self.name},
score={self.__security_score:.1f})'

def to_dict(self) -> dict:
    base = super().to_dict()
    base.update({'password_history': self.__password_history, 'security_score':
self.__security_score})
    return base

@classmethod
def from_dict(cls, data: dict) -> 'User':
    return cls(user_id=data['user_id'], name=data['name'], email=data['email'],
              password_history=data.get('password_history', []),
              security_score=data.get('security_score', 0.0))
```

3.4 admin.py — System Administrator (Inheritance + Polymorphism)

Purpose: Admin extends Person and adds system-level capabilities: generating reports and managing users. Its display_details() output differs from User's, demonstrating polymorphism.

OOP Concepts Demonstrated

- Inheritance — Admin extends Person and adds an auto-generated admin_id attribute (ADM-{user_id} if none provided).
- Polymorphism — display_details() renders an ADMIN PROFILE box with admin_id visible, contrasting with User's USER PROFILE box.
- Delegation — generate_reports() and manage_users() delegate to the system object, keeping the Admin class thin.
- Magic Methods — __str__ and __repr__ identify the admin with name and admin_id for debugging.

Full Source Code

```
# =====
# Admin - Derived from Person
# Cybersecurity Password Strength Analyzer
# =====

from models.person import Person

class Admin(Person):
    """
    System administrator with report-generation and user-management rights.
    Demonstrates: Inheritance, Polymorphism.
    """

    def __init__(self, user_id: str, name: str, email: str, admin_id: str = None):
        super().__init__(user_id, name, email)
        self.__admin_id = admin_id or f'ADM-{user_id}'

    @property
    def admin_id(self) -> str: return self.__admin_id

    def display_details(self) -> str:
        return (
            f'
            f'                ADMIN PROFILE
            f'                ||\n'
            f'  User ID : {self.user_id:<26}||\n'
            f'  Admin ID: {self.__admin_id:<26}||\n'
            f'  Name    : {self.name:<26}||\n'
            f'  Email   : {self.email:<26}||\n'
            f'                ||
        )

    def update_profile(self, **kwargs) -> None:
        if 'name' in kwargs: self.name = kwargs['name']
        if 'email' in kwargs: self.email = kwargs['email']

    def generate_reports(self, system) -> list: return system.generate_reports()
    def manage_users(self, system) -> list: return system.users
```

```
    def __str__(self) -> str: return f'Admin({self.name},
admin_id={self.__admin_id})'
    def __repr__(self) -> str: return f'Admin(user_id={self.user_id!r},
admin_id={self.__admin_id!r})'

    def to_dict(self) -> dict:
        base = super().to_dict()
        base['admin_id'] = self.__admin_id
        return base

    @classmethod
    def from_dict(cls, data: dict) -> 'Admin':
        return cls(user_id=data['user_id'], name=data['name'], email=data['email'],
                    admin_id=data.get('admin_id'))
```

3.5 password.py — Password Model (Encapsulation + Static Methods)

Purpose: Password represents a single password and performs fundamental analysis — rule checking, strength scoring, entropy calculation, and character breakdown. It is instantiated once per password submitted for analysis.

OOP Concepts Demonstrated

- Encapsulation — self.__password_text is private; external code gets only the masked version (first 2 chars + asterisks).
- Tuple (immutable) — RULES is a tuple of (description, lambda) pairs. Using a tuple signals that these rules are fixed contract terms, not modifiable data.
- Lambda functions — Each rule uses a lambda that takes the password string and returns a boolean.
- Static Method — calculate_entropy_static() takes a plain string and can be called without an instance, exposing the algorithm for reuse.
- Magic Methods — __len__ returns password length; __eq__ compares raw text for deduplication; __str__ shows the masked version.

Scoring Algorithm

Factor	Max Points	How Calculated
Password Length	30	$\min(30, \text{length} \times 2.5)$
Character Diversity	30	$4 \text{ types} \times 7.5 \text{ pts each (upper, lower, digit, special)}$
Entropy Bonus	20	$\min(20, \text{entropy} / 4)$
Unique Char Ratio	20	$(\text{unique chars} / \text{length}) \times 20$

Strength Levels

Score Range	Strength Level	Implication
0 – 20	Very Weak	Cracked instantly
21 – 40	Weak	Cracked in seconds
41 – 60	Moderate	Some resistance
61 – 80	Strong	Difficult to crack
81 – 100	Very Strong	Practically uncrackable

Full Source Code

```
# =====  
# Password Model  
# Cybersecurity Password Strength Analyzer  
# =====  
  
import re, math, string  
  
class Password:
```

```

"""
Represents a password and provides fundamental analysis.
Demonstrates: Encapsulation, Static Methods, Magic Methods.
"""

# Password rules stored as tuple (immutable)
RULES = (
    ('Minimum 8 characters', lambda p: len(p) >= 8),
    ('Contains uppercase letter', lambda p: bool(re.search(r'[A-Z]', p))),
    ('Contains lowercase letter', lambda p: bool(re.search(r'[a-z]', p))),
    ('Contains digit', lambda p: bool(re.search(r'\d', p))),
    ('Contains special character', lambda p: bool(re.search(r'[@#$$%^&*]',
p))),
)

STRENGTH_LEVELS = {
    (0, 20): 'Very Weak', (21, 40): 'Weak', (41, 60): 'Moderate',
    (61, 80): 'Strong', (81, 100): 'Very Strong',
}

def __init__(self, password_text: str):
    if not password_text: raise ValueError('Password cannot be empty.')
    self.__password_text = password_text
    self.__length = len(password_text)
    self.__strength = self.calculate_strength()

@property
def password_text(self) -> str: return self.__password_text
@property
def length(self) -> int: return self.__length
@property
def strength(self) -> str: return self.__strength

def analyze_password(self) -> dict:
    rules_passed, rules_failed = [], []
    for desc, fn in self.RULES:
        (rules_passed if fn(self.__password_text) else
rules_failed).append(desc)
    score = self.calculate_score()
    return { 'password_masked': self.__password_text[:2] + '*' * max(0,
self.__length-2),
            'length': self.__length, 'score': score,
            'strength': self._label_from_score(score),
            'rules_passed': rules_passed, 'rules_failed': rules_failed,
            'entropy': self.calculate_entropy(),
            'character_analysis': self._character_analysis() }

def calculate_score(self) -> float:
    score = min(30, self.__length * 2.5)
    has_upper = bool(re.search(r'[A-Z]', self.__password_text))
    has_lower = bool(re.search(r'[a-z]', self.__password_text))
    has_digit = bool(re.search(r'\d', self.__password_text))
    has_special = bool(re.search(r'^A-Za-z0-9]', self.__password_text))
    score += sum([has_upper, has_lower, has_digit, has_special]) * 7.5
    score += min(20, self.calculate_entropy() / 4)
    unique_ratio = len(set(self.__password_text)) / self.__length if
self.__length else 0
    score += unique_ratio * 20
    return round(min(100, max(0, score)), 1)

@staticmethod

```

```
def calculate_entropy_static(password_text: str) -> float:
    charset_size = 0
    if re.search(r'[a-z]', password_text): charset_size += 26
    if re.search(r'[A-Z]', password_text): charset_size += 26
    if re.search(r'\d', password_text): charset_size += 10
    if re.search(r'^A-Za-z0-9', password_text): charset_size += 32
    return round(len(password_text) * math.log2(charset_size), 2) if
charset_size else 0.0

def calculate_entropy(self) -> float:
    return Password.calculate_entropy_static(self.__password_text)

def _label_from_score(self, score: float) -> str:
    for (lo, hi), label in self.STRENGTH_LEVELS.items():
        if lo <= score <= hi: return label
    return 'Very Strong' if score > 80 else 'Very Weak'

def __str__(self) -> str:
    masked = self.__password_text[:2] + '*' * max(0, self.__length-2)
    return f'Password({masked}, strength={self.__strength})'
def __repr__(self) -> str: return f'Password(length={self.__length},
strength={self.__strength!r})'
def __len__(self) -> int: return self.__length
def __eq__(self, other) -> bool:
    return isinstance(other, Password) and self.__password_text ==
other.__password_text
```

3.6 password_analyzer.py — Deep Analysis Engine (Static + Class Methods)

Purpose: PasswordAnalyzer performs deep complexity analysis, dual-mode entropy calculation (Shannon + charset-based), and pattern detection. It is composed inside PasswordStrengthSystem and used in the full analysis pipeline.

OOP Concepts Demonstrated

- Composition — PasswordAnalyzer is a component owned by PasswordStrengthSystem (has-a relationship).
- Static Method — calculate_entropy() computes both Shannon and charset-based entropy from a plain string, with no instance state needed.
- Class Method & Class Attribute — _total_analyses is a class-level counter incremented by every full_analysis() call. get_total_analyses() and overall_security_statistics() are @classmethods that read it.
- Pattern Detection — detect_patterns() identifies repeated characters, sequential letters/numbers, keyboard walks, date patterns, and leet-speak substitutions with severity tags.
- Magic Methods — __str__ shows analysis ID and current score; __repr__ gives the ID for debugging.

Pattern Severity Reference

Pattern Type	Severity	Example
Keyboard Walk	Critical	qwerty, asdf, zxcv, 1qaz
Sequential Letters	High	abc, xyz, nop
Sequential Numbers	High	123, 456, 987
Repeated Characters	Medium	aaa, 111, !!! (3+ repeats)
Date Pattern	Medium	2001, 12/25/1990
Leet-Speak (2+ chars)	Low	p@\$\$w0rd (@ → a, 0 → o)

Full Source Code

```
# =====
# PasswordAnalyzer Model
# Cybersecurity Password Strength Analyzer
# =====

import re, math
from datetime import datetime

class PasswordAnalyzer:
    """
    Deep analysis engine for password complexity, entropy,
    and pattern detection.
    Demonstrates: Composition target, Static Methods, Class Methods.
    """

    _total_analyses = 0    # Class-level counter
```

```

def __init__(self, analysis_id: str = None):
    self.__analysis_id = analysis_id or
f'ANA-{datetime.now().strftime("%Y%m%d%H%M%S")}'
    self.__password_score = 0.0

@property
def analysis_id(self)    -> str:    return self.__analysis_id
@property
def password_score(self) -> float: return self.__password_score

def check_complexity(self, password_text: str) -> dict:
    length      = len(password_text)
    has_upper   = bool(re.search(r'[A-Z]', password_text))
    has_lower   = bool(re.search(r'[a-z]', password_text))
    has_digit   = bool(re.search(r'\d', password_text))
    has_special = bool(re.search(r'^A-Za-z0-9', password_text))
    char_variety = sum([has_upper, has_lower, has_digit, has_special])
    unique_chars = len(set(password_text))
    return { 'length': length, 'has_uppercase': has_upper, 'has_lowercase':
has_lower,
            'has_digits': has_digit, 'has_special_characters': has_special,
            'character_variety_score': char_variety, 'unique_characters':
unique_chars,
            'unique_ratio': round(unique_chars/length, 2) if length else 0,
            'meets_minimum_length': length >= 8,
            'complexity_rating': self.rate_complexity(char_variety, length,
unique_chars/length if length else 0) }

@staticmethod
def calculate_entropy(password_text: str) -> dict:
    # Shannon entropy + charset-based entropy
    freq = {}
    for ch in password_text: freq[ch] = freq.get(ch, 0) + 1
    length = len(password_text)
    shannon = -sum((c/length)*math.log2(c/length) for c in freq.values() if c >
0)

    charset = sum([26 if re.search(r'[a-z]',p := password_text) else 0,
                    26 if re.search(r'[A-Z]',p) else 0,
                    10 if re.search(r'\d',p) else 0,
                    32 if re.search(r'^A-Za-z0-9',p) else 0])
    charset_entropy = length * math.log2(charset) if charset else 0
    return { 'shannon_entropy': round(shannon, 4), 'charset_entropy':
round(charset_entropy, 2),
            'charset_size': charset, 'total_bits': round(charset_entropy, 2) }

def detect_patterns(self, password_text: str) -> dict:
    patterns_found = []
    if re.findall(r'(\.)\1{2,}', password_text):
        patterns_found.append({'type': 'Repeated Characters', 'severity':
'Medium'})
    if self._find_sequential_alpha(password_text):
        patterns_found.append({'type': 'Sequential Letters', 'severity':
'High'})
    if self._find_sequential_numbers(password_text):
        patterns_found.append({'type': 'Sequential Numbers', 'severity':
'High'})
    if self._find_keyboard_patterns(password_text):
        patterns_found.append({'type': 'Keyboard Patterns', 'severity':
'Critical'})
    return { 'patterns_detected': len(patterns_found), 'patterns':
patterns_found,
            'is_pattern_free': len(patterns_found) == 0 }

```

```
def full_analysis(self, password_text: str) -> dict:
    PasswordAnalyzer._total_analyses += 1
    complexity = self.check_complexity(password_text)
    entropy = self.calculate_entropy(password_text)
    patterns = self.detect_patterns(password_text)
    score = self._composite_score(complexity, entropy, patterns)
    self.__password_score = score
    return { 'analysis_id': self.__analysis_id, 'complexity': complexity,
            'entropy': entropy, 'patterns': patterns, 'score': score,
            'strength': self._strength_label(score),
            'analyzed_at': datetime.now().strftime('%Y-%m-%d %H:%M:%S') }

    @classmethod
    def get_total_analyses(cls) -> int: return cls._total_analyses

    @classmethod
    def overall_security_statistics(cls) -> dict:
        return { 'total_analyses_performed': cls._total_analyses }

    def __str__(self) -> str: return f'PasswordAnalyzer(id={self.__analysis_id},
score={self.__password_score}) '
    def __repr__(self) -> str: return
f'PasswordAnalyzer(analysis_id={self.__analysis_id!r}) '
```

3.7 vulnerability_scanner.py — Vulnerability Scanner

Purpose: VulnerabilityScanner scans passwords against curated lists of common passwords, dictionary words, and keyboard-walk patterns. It returns a risk report with severity-tagged findings.

OOP Concepts Demonstrated

- Class Attributes (Set) — COMMON_PASSWORDS, DICTIONARY_WORDS, and KEYBOARD_PATTERNS are class-level sets, giving O(1) membership lookup — much faster than list iteration.
- Encapsulation — self.__detected_risks is reset on every scan call, preventing state bleed between scans.
- Risk Severity Model — Each detected risk is tagged Critical, High, Medium, or Low. _calculate_risk_level() picks the highest severity across all findings.
- Magic Methods — __str__ shows current risk count; __repr__ shows the size of the common password database.

Risk Severity Reference

Severity	Trigger	Example
Critical	Common Password / Keyboard Walk	"password", "qwerty", "admin123"
High	Sequential Pattern	"abc", "123", "xyz"
Medium	Dictionary Words / Repeated Chars	"love", "aaa", date patterns

Full Source Code

```
# =====
# VulnerabilityScanner Model
# Cybersecurity Password Strength Analyzer
# =====

import re

class VulnerabilityScanner:
    """
    Scans passwords against known weak patterns, dictionary words,
    and common password lists.
    """

    # Common weak passwords - stored as set for O(1) lookup
    COMMON_PASSWORDS = {
        'password', '123456', '123456789', '12345678', 'qwerty',
        'abc123', '111111', 'iloveyou', 'sunshine', 'admin',
        'welcome', 'monkey', 'dragon', 'letmein', 'football',
        'password123', 'qwerty123', 'p@ssw0rd', 'pass@123',
        # ... (50 entries total)
    }

    DICTIONARY_WORDS = {
        'love', 'life', 'time', 'world', 'money', 'school', 'summer',
        'winter', 'computer', 'security', 'network', 'server',
```

```

        # ... (40 entries total)
    }

    KEYBOARD_PATTERNS = {
        'qwerty', 'asdfg', 'zxcvb', 'qazwsx', 'lqaz',
        '2wsx', '3edc', 'poiuy', 'mnbvc', '!@#$',
    }

    def __init__(self):
        self.__weak_password_list = list(self.COMMON_PASSWORDS)
        self.__detected_risks = []

    @property
    def weak_password_list(self) -> list: return list(self.__weak_password_list)
    @property
    def detected_risks(self) -> list: return list(self.__detected_risks)

    def scan_password(self, password_text: str) -> dict:
        self.__detected_risks = []
        is_common = self._check_common_password(password_text)
        dict_words = self._check_dictionary_words(password_text)
        repeated = self._check_repeated_characters(password_text)
        sequential = self._check_sequential_patterns(password_text)
        keyboard = self._check_keyboard_patterns(password_text)
        return { 'is_common_password': is_common,
                  'dictionary_words_found': dict_words,
                  'repeated_characters': repeated,
                  'sequential_patterns': sequential,
                  'keyboard_patterns': keyboard,
                  'total_risks': len(self.__detected_risks),
                  'risk_level': self._calculate_risk_level(),
                  'risks': list(self.__detected_risks) }

    def _check_common_password(self, password_text: str) -> bool:
        lower = password_text.lower().strip()
        if lower in self.COMMON_PASSWORDS:
            self.__detected_risks.append({'risk': 'Common Password', 'severity':
'Critical',
            'detail': 'Appears in common password lists; cracked instantly.'})
            return True
        for suffix in ['!', '1', '123', '@', '#']:
            if lower.rstrip(suffix) in self.COMMON_PASSWORDS:
                self.__detected_risks.append({'risk': 'Common Password Variant',
                'severity': 'High', 'detail': f'Trivial variation (suffix
{suffix!r})'})
            return True
        return False

    def _calculate_risk_level(self) -> str:
        if not self.__detected_risks: return 'Low'
        scores = {'Critical': 4, 'High': 3, 'Medium': 2, 'Low': 1}
        m = max(scores.get(r['severity'], 0) for r in self.__detected_risks)
        return {4: 'Critical', 3: 'High', 2: 'Medium'}.get(m, 'Low')

    def __str__(self) -> str: return
f'VulnerabilityScanner(risks_detected={len(self.__detected_risks)})'
    def __repr__(self) -> str: return
f'VulnerabilityScanner(common_passwords={len(self.COMMON_PASSWORDS)})'

```

3.8 password_generator.py — Password Generator

Purpose: PasswordGenerator creates cryptographically-inspired random passwords and word-based passphrases. It guarantees at least one character from each selected category before shuffling, so all generated passwords meet basic complexity requirements.

OOP Concepts Demonstrated

- Encapsulation — self.__generated_password stores the last generated password privately; exposed via read-only @property.
- Algorithm Design — The 'required list' approach ensures category coverage (at least 1 uppercase, 1 lowercase, 1 digit, 1 special char), then fills the rest randomly before shuffling.
- Method Variants — generate_password() for configurable random passwords; generate_passphrase() for human-memorable word-based combinations; suggest_password() returns 5 options at varying lengths.
- Magic Methods — __str__ shows a masked preview (first 3 chars + ***); __repr__ identifies the class.

Generation Modes

Mode	Method	Description
Random Password	generate_password()	Configurable length (8–64), toggle uppercase/lowercase/digits/specials
Passphrase	generate_passphrase()	Combines 3–6 random words with random separators and digit suffixes (e.g. Storm42!Raven7)
5 Suggestions	suggest_password()	Returns list of 5 random passwords at lengths 12, 14, 16, 18, 20

Full Source Code

```
# =====
# PasswordGenerator Model
# Cybersecurity Password Strength Analyzer
# =====

import random, string

class PasswordGenerator:
    """Generates strong, random passwords and provides suggestions."""

    def __init__(self):
        self.__generated_password = ''

    @property
    def generated_password(self) -> str: return self.__generated_password

    def generate_password(self, length: int = 16, use_upper: bool = True,
                        use_lower: bool = True, use_digits: bool = True,
                        use_special: bool = True) -> str:
        if length < 8: length = 8
        charset, required = '', []
```

```

        if use_upper: charset += string.ascii_uppercase;
        required.append(random.choice(string.ascii_uppercase))
        if use_lower: charset += string.ascii_lowercase;
        required.append(random.choice(string.ascii_lowercase))
        if use_digits: charset += string.digits;
        required.append(random.choice(string.digits))
        if use_special:
            sp = '!@#$$%^&*()-_+[]{}|;:.,.<>?'
            charset += sp; required.append(random.choice(sp))
        if not charset: charset = string.ascii_letters + string.digits
        chars = required + [random.choice(charset) for _ in range(length -
len(required))]
        random.shuffle(chars)
        self.__generated_password = ''.join(chars)
        return self.__generated_password

def suggest_password(self, base_password: str = '') -> list:
    return [self.generate_password(length=l) for l in [12, 14, 16, 18, 20]]

def generate_passphrase(self, word_count: int = 4) -> str:
    word_bank = ['Alpha', 'Bravo', 'Cyber', 'Delta', 'Eagle', 'Frost', 'Guard',
        'Hydra', 'Intel', 'Joker', 'Knife', 'Lunar', 'Nexus', 'Omega',
        'Prism', 'Quake', 'Raven', 'Storm', 'Titan', 'Ultra']
    sep = random.choice(['!', '@', '#', '$', '-', '_', '.', '&'])
    words = random.sample(word_bank, min(word_count, len(word_bank)))
    parts = [w + str(random.randint(0, 99)) for w in words]
    self.__generated_password = sep.join(parts)
    return self.__generated_password

def __str__(self) -> str:
    masked = self.__generated_password[:3] + '***' if self.__generated_password
else ' (none) '
    return f'PasswordGenerator(last={masked})'

def __repr__(self) -> str: return 'PasswordGenerator()'

```

3.9 security_report.py — Security Report Model

Purpose: SecurityReport generates, stores, and exports formatted text reports from password analysis results. It maintains a class-level counter (_report_counter) to auto-assign sequential report IDs (RPT-0001, RPT-0002, ...).

OOP Concepts Demonstrated

- Class Attribute & Auto-ID — _report_counter is a class-level integer incremented in __init__, giving every report a unique sequential ID without external input.
- Encapsulation — report_id, report_date, security_level, and report_data are all private. The security_level setter validates against a whitelist of 6 allowed levels.
- Magic Methods — __str__ gives a one-line summary (ID, date, level); __len__ returns the number of data points in report_data.
- Serialization — export_report() returns a plain dict for JSON storage, enabling reports to be persisted and reloaded across sessions.

Report Structure

- Header — Report ID, generation date, and username.
- Overall Result — Score (0–100) and Strength Label.
- Complexity Analysis — Length, character types (upper/lower/digit/special), variety score, unique character ratio, and complexity rating.
- Entropy Analysis — Shannon entropy, charset-based entropy, total bits, and charset size.
- Pattern Detection — Each detected pattern listed with its severity tag.
- Vulnerability Scan — Risk level, common password flag, dictionary words found, and all risk details.

Full Source Code

```
# =====
# SecurityReport Model
# Cybersecurity Password Strength Analyzer
# =====

from datetime import datetime

class SecurityReport:
    """
    Represents a security report generated from password analyses.
    """

    _report_counter = 0    # Class attribute - auto-incrementing ID

    def __init__(self, report_id: str = None, security_level: str = 'Unknown',
report_data: dict = None):
        SecurityReport._report_counter += 1
        self.__report_id    = report_id or
f'RPT-{SecurityReport._report_counter:04d}'
        self.__report_date  = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        self.__security_level = security_level
        self.__report_data  = report_data or {}

    @property
```

```

def report_id(self)    -> str: return self.__report_id
@property
def report_date(self) -> str: return self.__report_date
@property
def security_level(self) -> str: return self.__security_level
@security_level.setter
def security_level(self, value: str):
    valid = ['Very Weak', 'Weak', 'Moderate', 'Strong', 'Very Strong', 'Unknown']
    self.__security_level = value if value in valid else 'Unknown'

def generate_report(self, analysis_result: dict, user_name: str = 'Anonymous')
-> str:
    self.__report_data    = analysis_result
    self.__security_level = analysis_result.get('strength', 'Unknown')
    score                 = analysis_result.get('score', 0)
    complexity            = analysis_result.get('complexity', {})
    entropy               = analysis_result.get('entropy', {})
    patterns              = analysis_result.get('patterns', {})
    vulnerabilities       = analysis_result.get('vulnerabilities', {})

    lines = [
        '=' * 55,
        '        PASSWORD SECURITY ANALYSIS REPORT',
        '=' * 55,
        f' Report ID    : {self.__report_id}',
        f' Date          : {self.__report_date}',
        f' User           : {user_name}',
        f' Score          : {score}/100',
        f' Strength       : {self.__security_level}',
        '-' * 55,
        ' COMPLEXITY    : ' + str(complexity.get('complexity_rating')),
        ' ENTROPY BITS   : ' + str(entropy.get('total_bits')),
        ' PATTERNS      : ' + str(patterns.get('patterns_detected')),
        '=' * 55,
    ]
    return '\n'.join(lines)

def export_report(self) -> dict:
    return { 'report_id': self.__report_id, 'report_date': self.__report_date,
            'security_level': self.__security_level, 'report_data':
self.__report_data }

def __str__(self) -> str:
    return f'SecurityReport(id={self.__report_id},
level={self.__security_level})'
def __repr__(self) -> str:
    return f'SecurityReport(report_id={self.__report_id!r})'
def __len__(self) -> int: return len(self.__report_data)

```

3.10 password_strength_system.py — Composition Root & Central Orchestrator

Purpose: PasswordStrengthSystem is the central orchestrator — the single entry point for all analysis operations. It demonstrates Composition by owning instances of PasswordAnalyzer, VulnerabilityScanner, and PasswordGenerator. It manages all registered Users, all analysis records, and all SecurityReports.

OOP Concepts Demonstrated

- Composition (has-a) — The system privately owns PasswordAnalyzer, VulnerabilityScanner, and PasswordGenerator. It does NOT inherit from them; it delegates to them.
- Data Structures — Uses dict (user_id → User) for O(1) user lookup; list for ordered analysis records; list for SecurityReport objects.
- Recursion — search_history() uses recursive index stepping: it checks record[index], then recursively calls itself with index+1. Base case is index >= len(records).
- Persistence — save_data() serializes users, records, and reports to JSON files. load_data() deserializes them back, reconstructing User and SecurityReport objects.
- Delegation — generate_strong_password(), suggest_passwords(), generate_passphrase() delegate to the internal PasswordGenerator instance.

Analysis Pipeline (6 Steps)

1. Step 1 — Create a Password object; run basic rule checks and character analysis.
2. Step 2 — Call PasswordAnalyzer.full_analysis() for complexity, entropy, and pattern detection.
3. Step 3 — Call VulnerabilityScanner.scan_password() for common password, dictionary, and keyboard pattern checks.
4. Step 4 — Combine all results into a single dict with crack time estimate and actionable recommendations.
5. Step 5 — Append the combined record to the analysis_records list.
6. Step 6 — If a user_id is provided and exists, attach the record to that user's password history.

Recursive History Search

```
def search_history(self, keyword, records=None, index=0) -> list:
    if records is None: records = self.__analysis_records
    if index >= len(records): return [] # Base case
    results = []
    if keyword.lower() in json.dumps(records[index]).lower():
        results.append(records[index])
    results.extend(self.search_history(keyword, records, index + 1)) # Recursive call
    return results
```

Full Source Code

```
# =====
# PasswordStrengthSystem - Main System (Composition Root)
# Cybersecurity Password Strength Analyzer
# =====
```

```

import os, json
from datetime import datetime
from models.password import Password
from models.password_analyzer import PasswordAnalyzer
from models.vulnerability_scanner import VulnerabilityScanner
from models.password_generator import PasswordGenerator
from models.security_report import SecurityReport
from models.user import User

class PasswordStrengthSystem:
    """
    Central system that composes all components.
    Demonstrates: Composition - contains Password, PasswordAnalyzer,
                  VulnerabilityScanner, PasswordGenerator, SecurityReport.
    """

    DATA_DIR = os.path.join(os.path.dirname(...), 'data')
    REPORTS_DIR = os.path.join(os.path.dirname(...), 'reports')
    EXPORTS_DIR = os.path.join(os.path.dirname(...), 'exports')

    def __init__(self):
        # Composition - system owns these components
        self.__analyzer = PasswordAnalyzer()
        self.__scanner = VulnerabilityScanner()
        self.__generator = PasswordGenerator()
        self.__users = {} # dict: user_id -> User
        self.__analysis_records = [] # list of analysis result dicts
        self.__reports = [] # list of SecurityReport objects

    @property
    def users(self) -> dict: return dict(self.__users)
    @property
    def analysis_records(self) -> list: return list(self.__analysis_records)
    @property
    def reports(self) -> list: return list(self.__reports)

    def register_user(self, user_id, name, email) -> User:
        if user_id in self.__users:
            raise ValueError(f'User ID {user_id!r} already exists.')
        user = User(user_id=user_id, name=name, email=email)
        self.__users[user_id] = user
        return user

    def analyze_password(self, password_text: str, user_id: str = None) -> dict:
        pwd = Password(password_text)
        basic_analysis = pwd.analyze_password()
        analyzer_result = self.__analyzer.full_analysis(password_text)
        vuln_result = self.__scanner.scan_password(password_text)
        combined = { 'score': analyzer_result['score'],
                     'strength': analyzer_result['strength'],
                     'complexity': analyzer_result['complexity'],
                     'entropy': analyzer_result['entropy'],
                     'patterns': analyzer_result['patterns'],
                     'vulnerabilities': vuln_result,
                     'crack_time_estimate':
self._estimate_crack_time(analyzer_result['entropy']),
                     'recommendations': self._generate_recommendations(
                         analyzer_result, vuln_result, basic_analysis),
                     'analyzed_at': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
                     'analysis_id': analyzer_result['analysis_id'] }
        self.__analysis_records.append(combined)
        if user_id and user_id in self.__users:

```

```
        self.__users[user_id].analyze_password(password_text, combined)
    return combined

def search_history(self, keyword, records=None, index=0) -> list:
    # Recursive search - demonstrates Recursion
    if records is None: records = self.__analysis_records
    if index >= len(records): return [] # base case
    results = []
    if keyword.lower() in json.dumps(records[index]).lower():
        results.append(records[index])
    results.extend(self.search_history(keyword, records, index + 1))
    return results

def save_data(self) -> str:
    # Serialize users, records, reports to JSON files
    ...

def load_data(self) -> str:
    # Deserialize from JSON files into memory
    ...

def __str__(self) -> str:
    return (f'PasswordStrengthSystem(users={len(self.__users)}, '
            f'analyses={len(self.__analysis_records)}, '
            f'reports={len(self.__reports)})')
def __repr__(self) -> str: return 'PasswordStrengthSystem()'
```

3.11 test_project.py — Smoke Test Suite

Purpose: test_project.py is a comprehensive smoke test that validates every major feature of the system in a single run. It covers all 14 test groups and terminates with a clear PASS/FAIL summary.

Full Source Code

```
"""Quick smoke test for all project modules."""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# — Test 1: All imports —————
from models import *; from services import *; from utils import *
print('[OK] All imports successful')

# — Test 2: Password model —————
from models.password import Password
p = Password('Test@123!Secure')
r = p.analyze_password()
print(f'[OK] Score: {r["score"]}, Strength: {r["strength"]}')
print(f'[OK] __len__: {len(p)}, __str__: {p}')

# — Test 3: PasswordAnalyzer —————
from models.password_analyzer import PasswordAnalyzer
a = PasswordAnalyzer()
full = a.full_analysis('Test@123!Secure')
print(f'[OK] Complexity Rating: {full["complexity"]["complexity_rating"]}')
print(f'[OK] Total analyses (class method): {PasswordAnalyzer.get_total_analyses()}')

# — Test 4: VulnerabilityScanner —————
from models.vulnerability_scanner import VulnerabilityScanner
v = VulnerabilityScanner()
scan = v.scan_password('password123')
print(f'[OK] Common detected: {scan["is_common_password"]}, Risks: {scan["total_risks"]}')

# — Test 5: PasswordGenerator —————
from models.password_generator import PasswordGenerator
g = PasswordGenerator()
print(f'[OK] Generated: {g.generate_password(16)} (length 16)')
print(f'[OK] Passphrase: {g.generate_passphrase(4)}')
print(f'[OK] Suggestions: {len(g.suggest_password())} passwords')

# — Test 6: PasswordStrengthSystem (Composition) —————
from models.password_strength_system import PasswordStrengthSystem
s = PasswordStrengthSystem()
u = s.register_user('T001', 'Test User', 'test@test.com')
res = s.analyze_password('Str0ng!P@ssw0rd#2026', user_id='T001')
print(f'[OK] Score: {res["score"]}, Crack time: {res["crack_time_estimate"]["human_readable"]}')

# — Test 7: Polymorphism —————
from models.user import User; from models.admin import Admin
for p in [User('P1', 'Alice', 'a@t.com'), Admin('P2', 'Bob', 'b@t.com')]:
    p.display_details()
print('[OK] Polymorphism: display_details() works for User and Admin')

# — Test 8: SecurityReport —————
```

```
from models.security_report import SecurityReport
rpt = SecurityReport()
print(f'[OK] SecurityReport: {rpt.report_id}, __len__: {len(rpt)}')

# — Test 9: Services - List Comprehensions & Lambdas —————
from services.analysis_service import AnalysisService
asvc = AnalysisService(s)
print(f'[OK] Weak: {len(asvc.get_weak_passwords())}, Strong: {len(asvc.get_strong_passwords())}')
print(f'[OK] Sorted by score: {len(asvc.sort_by_score())} records')

# — Test 10: Generator —————
from services.report_service import ReportService
rsvc = ReportService(s)
for rpt_data in rsvc.report_generator(): print(f'[OK] Generator: {rpt_data["report_id"]}')

# — Test 11: Recursive search —————
print(f'[OK] Recursive search: {len(s.search_history("Strong"))} result(s)')

# — Test 12: Save & Load —————
print(f'[OK] {s.save_data()}')
s2 = PasswordStrengthSystem(); s2.load_data()
print(f'[OK] Loaded users: {len(s2.users)}, records: {len(s2.analysis_records)}')

# — Test 13: CSV Export —————
from services.export_service import ExportService
esvc = ExportService(s)
print(f'[OK] CSV exports: {esvc.export_security_reports_csv()}')

# — Test 14: Custom exceptions —————
from utils.exceptions import EmptyPasswordError, WeakPasswordError
try: raise EmptyPasswordError()
except EmptyPasswordError as e: print(f'[OK] EmptyPasswordError: {e}')

print('\n' + '='*55)
print(' ALL TESTS PASSED SUCCESSFULLY!')
print('='*55)
```

4. Test Results (test_project.py)

The project includes a comprehensive smoke-test file that validates every major feature. All 14 test groups pass successfully.

#	Test Group	What Is Verified
1	Imports	All modules (models, services, utils) import without errors
2	Password Model	Score, strength, <code>__len__</code> , <code>__str__</code> , <code>__repr__</code> all work correctly
3	PasswordAnalyzer	Complexity, entropy, patterns, static method, class method
4	VulnerabilityScanner	Common password detection, keyboard pattern detection
5	PasswordGenerator	Random password length validation, passphrase format, 5 suggestions
6	PasswordStrengthSystem	Full pipeline: user register, analyze, crack time, recommendations
7	Polymorphism	<code>display_details()</code> works and differs for both User and Admin instances
8	SecurityReport	Report generation, RPT-xxxx sequential ID, <code>__len__</code> magic method
9	Services	List comprehensions: weak/strong/risky filtering; lambda sort by score and risk
10	Generator	<code>report_generator()</code> yields report dicts one at a time
11	Recursion	<code>search_history('Strong')</code> finds all matching records recursively
12	Save & Load	JSON persistence round-trip: save → new system instance → load → verify
13	CSV Export	All three CSV files generated to exports/ directory successfully
14	Custom Exceptions	<code>EmptyPasswordError</code> and <code>WeakPasswordError</code> raise and display correctly

All 14 test groups pass. The application handles user registration, multi-factor password analysis, vulnerability detection, strong password generation, report creation, and complete data persistence through an intuitive terminal interface.

5. Python Features Used

Feature	File(s)	Example / Notes
Abstract Base Class (ABC)	person.py	from abc import ABC, abstractmethod
@property / @setter	All model files	@property def name: ...; @name.setter with validation
Static Method	password.py, password_analyzer.py	@staticmethod def calculate_entropy(text)
Class Method	password_analyzer.py, security_report.py	@classmethod def get_total_analyses(cls)
Magic Methods	All classes	__len__, __eq__, __str__, __repr__ implemented
Name-mangling (double __)	All classes	self.__password_text — true private storage
Recursion	password_strength_system.py	search_history(keyword, records, index+1)
Lambda Functions	password.py	RULES = (('Min 8', lambda p: len(p)>=8), ...)
List Comprehensions	analysis_service.py	[r for r in records if r['score'] < 40]
Generator Function	report_service.py	yield report_data — lazy report iteration
Custom Exceptions	utils/exceptions.py	class EmptyPasswordError(Exception)
JSON Serialization	password_strength_system.py	json.dump / json.load with to_dict() / from_dict()
CSV Export	export_service.py	csv.writer with DictWriter and fieldnames
Decorators	services/	@log_analysis / @validate_input on service methods
Set (O(1) lookup)	vulnerability_scanner.py	COMMON_PASSWORDS = {'password', '123456', ...}
Tuple (immutable)	password.py	RULES = (('Name', fn), ...) — immutable contract

6. Cybersecurity Features

6.1 Password Validation Rules

Rule	Implementation
Minimum 8 Characters	<code>len(password) >= 8</code> — enforced in Password.RULES tuple
Uppercase Letter Check	<code>re.search(r'[A-Z]', password)</code> — regex match
Lowercase Letter Check	<code>re.search(r'[a-z]', password)</code> — regex match
Number Check	<code>re.search(r'\d', password)</code> — regex match
Special Character Check	<code>re.search(r'![@#\$\$%^&*...]', password)</code> — regex match

6.2 Crack Time Estimation

The system estimates password-cracking difficulty by computing the total number of possible combinations from the entropy bits, then dividing by the assumed attack speed of 10 billion guesses per second (modern GPU cluster scenario).

Crack Time	Difficulty Rating	Entropy Range
Instant	Trivial	< 1 bit
< 1 second	Very Easy	1–33 bits
Minutes	Easy	34–43 bits
Hours	Moderate	44–50 bits
Days	Hard	51–58 bits
Years	Very Hard	59–75 bits
10 ⁶ + years	Extremely Hard	76–100 bits
~10 ^N years	Practically Impossible	> 100 bits

6.3 Sample Analysis Output

Example: Password Anuj@1234

Metric	Result
Length	9 characters
Uppercase	Yes
Lowercase	Yes
Digits	Yes
Special Characters	Yes
Strength Score	85 / 100
Security Level	Very Strong
Estimated Crack Difficulty	High

7. Conclusion

This workshop project successfully demonstrates all four pillars of Object-Oriented Programming through a real-world cybersecurity application:

OOP Pillar	How It Is Demonstrated
Abstraction	Person defines the contract via <code>@abstractmethod</code> ; subclasses fill in the details. Person cannot be instantiated directly.
Encapsulation	Every sensitive attribute (password text, user history, analysis score) is private via double-underscore, accessed only through <code>@property</code> decorators with validation.
Inheritance	User and Admin both extend Person without duplicating common logic. Both call <code>super().__init__()</code> and add only their own role-specific attributes and methods.
Polymorphism	The same <code>display_details()</code> call produces a USER PROFILE box for a User and an ADMIN PROFILE box for an Admin — different behaviour from the same interface.

Beyond the four pillars, the project showcases advanced Python features: static and class methods, magic methods, generators, list comprehensions, lambda functions, decorators, recursion, custom exceptions, and JSON/CSV data persistence — making it a thorough exercise in practical Python programming applied to real-world security.

Project Outcome: All 14 test groups pass. The application handles user registration, multi-factor password analysis, vulnerability detection, strong password generation, report creation, and complete data persistence through an intuitive terminal interface.

—— End of Report ——

OOP Python Workshop • NIET Greater Noida • June 2026